

A URL Shortener

by

TAN Xingzhi

3036532736

Dissertation midterm report

2025/2026 Sem 2

Master of Engineering

In

EEE

of

The University of Hong Kong

CONTENTS

1.	Project Overview	3
1.1	Background	3
1.2	Project objectives	4
1.3	Original research question	4
2.	Progress Update	5
2.1	User Authentication & Account Management	5
2.2	URL Shortening Core Engine	6
2.3	Expiration & Lifecycle Management	7
2.4	Redirection & Click Analytics	7
2.5	User Dashboard & Link Management	8
2.6	Detailed Statistics View	8
2.7	QR Code Generation	9
2.8	Administrative Panel	10
2.9	Cross-Database Compatibility	10
2.10	Frontend User Interface	11
2.11	Deployment Readiness	11
3.	Methodology & Implementation	11
3.1	Database Design	11
3.2	Short Code Generation & Validation Algorithm	14
3.3	QR Code Integration	17
4.	Critical Evaluation	17
4.1	Security Weaknesses	17
4.2	Usability and Functional Gaps	19
4.3	Summary	20
5.	Conclusion & Next Steps.....	21
5.1	Conclusion	21
5.2	Next Steps	21
5.3	Anticipated Final Deliverables	24

1. Project Overview

1.1 Background

URL shortening services are online services or web tools that can convert lengthy Uniform Resource Locators (URLs) into short and easy-to-remember URLs. This reduces the complexity for users to remember and type URLs and facilitates sharing links in scenarios with character limits, such as social media and SMS. The core function of shortening links is to reduce the number of characters in the original URL, thereby improving dissemination efficiency; its principles include URL mapping mechanisms and HTTP redirection technology.

The core value of shortening link services lies not only in improving the readability and user experience of links, but also in their powerful data analysis and click tracking capabilities—providing data such as click volume, user geolocation, device type, and conversion rate to help businesses optimize marketing campaigns and analyze user behavior. It has been deeply integrated into the digital business ecosystem.

The earliest URL shortening service can be traced back to TinyURL, developed by Kevin Gilbertson in 2002. Its predecessor can be traced back to 2001. The birth of TinyURL influenced the creation of at least 100 subsequent URL shortening services [1]. According to The New York Times, TinyURL was founded in 2002 by an amateur named Kevin Gilbertson and was once the most popular URL shortening service. TinyURL later grew rapidly and its database stored about 293 million URLs. Modern enterprise-level link shortening services have long gone beyond basic redirection functions, with more extended functions such as providing data tracking functions such as click volume, region, device and source channel. They can generate dynamic links with UTM parameters; at the same time, they also support brand customization, allowing users to replace public domains with their own domains (e.g., s.companyname.com/event), thereby increasing the company's exposure, achieving the purpose of publicity, and increasing the click-through rate by an average of 34%; in terms of security, they can block phishing/spam websites and prevent links from

being blocked; they support link expiration dates, password access and one-time use restrictions. They also have dynamic adjustment functions—if printed materials have been sent and the original link is found to be invalid, the target URL pointed to by the shortened link can be modified in the backend, so that the materials do not need to be reprinted[2].

1.2 Project objectives

Build a fully functional short link generation and management platform, providing features such as custom short codes, access statistics, expiration control, and QR code generation, while also having the ability to analyze multiple types of click data.

- Implements user registration, login, and password reset authentication functions.
- Supports shortening long URLs to system-generated or user-defined short codes.
- Records detailed data for each short link access (IP, User-Agent, Referer).
- Provides flexible expiration time settings (1 day/7 days/30 days/permanent) and custom expiration conditions.
- Generates a QR code for each short link and supports downloading.
- Supports user management of their generated short codes. Provides short link access data analysis and visualization, generating analysis reports for user-generated short code click data.
- User-friendly interface

1.3 Original research question

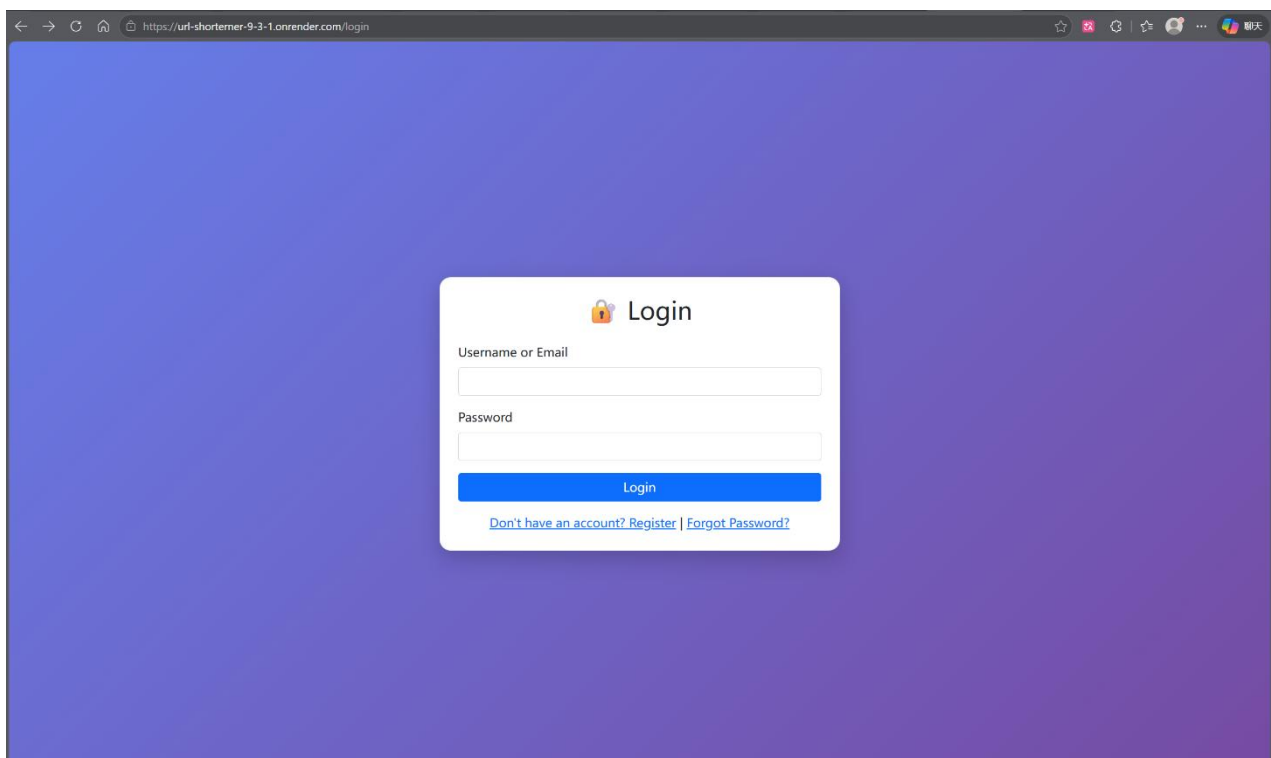
- How to design a lightweight, efficient, secure, and stable URL shortener to achieve fast compression and accurate redirection of long URLs, while addressing the core pain points of traditional shorteners such as redirection delays and broken links?
- Regarding the uniqueness conflicts and data security issues during URL shortening, what encoding algorithms and storage schemes should be adopted to ensure the uniqueness of the shortened URL while protecting user privacy and link security?
- How to add practical extended functions to the URL shortener (such as access

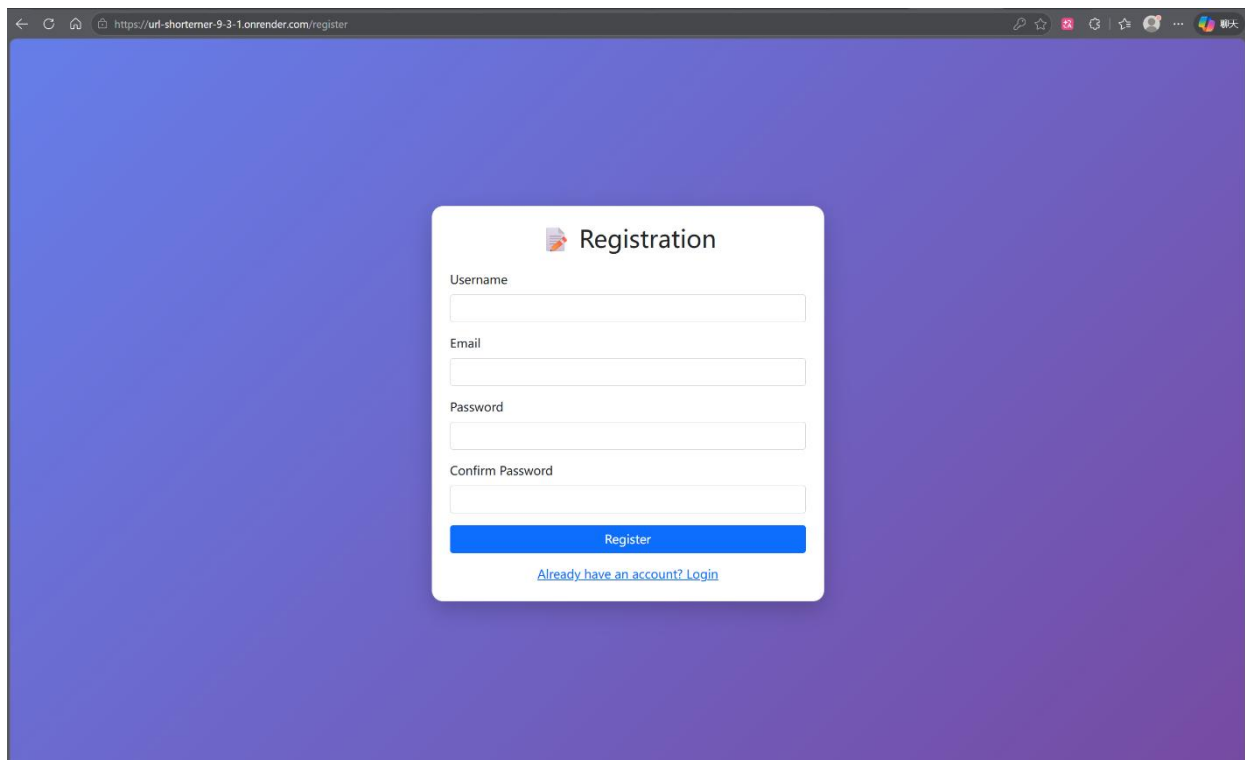
statistics, expiration management, and custom shortening codes) to improve product usability and meet the actual needs of individuals and small teams?

2. Progress Update

2.1 User Authentication & Account Management

- **User Registration:** New users can create an account by providing a unique username, email, and password (minimum 6 characters). Passwords are securely hashed by the `generate_password_hash`.
- **User login:** User authentication uses both username and email address as identifiers. Session management is performed using Flask's signed cookies.
- **Password Reset Process:** A complete "Forgot Password" workflow has been implemented. The system generates a UUID-based reset token valid for 1 hour and stores it in the database. A reset link will be displayed in the server console.
- **Logout:** Clears the session and redirects to the login page.



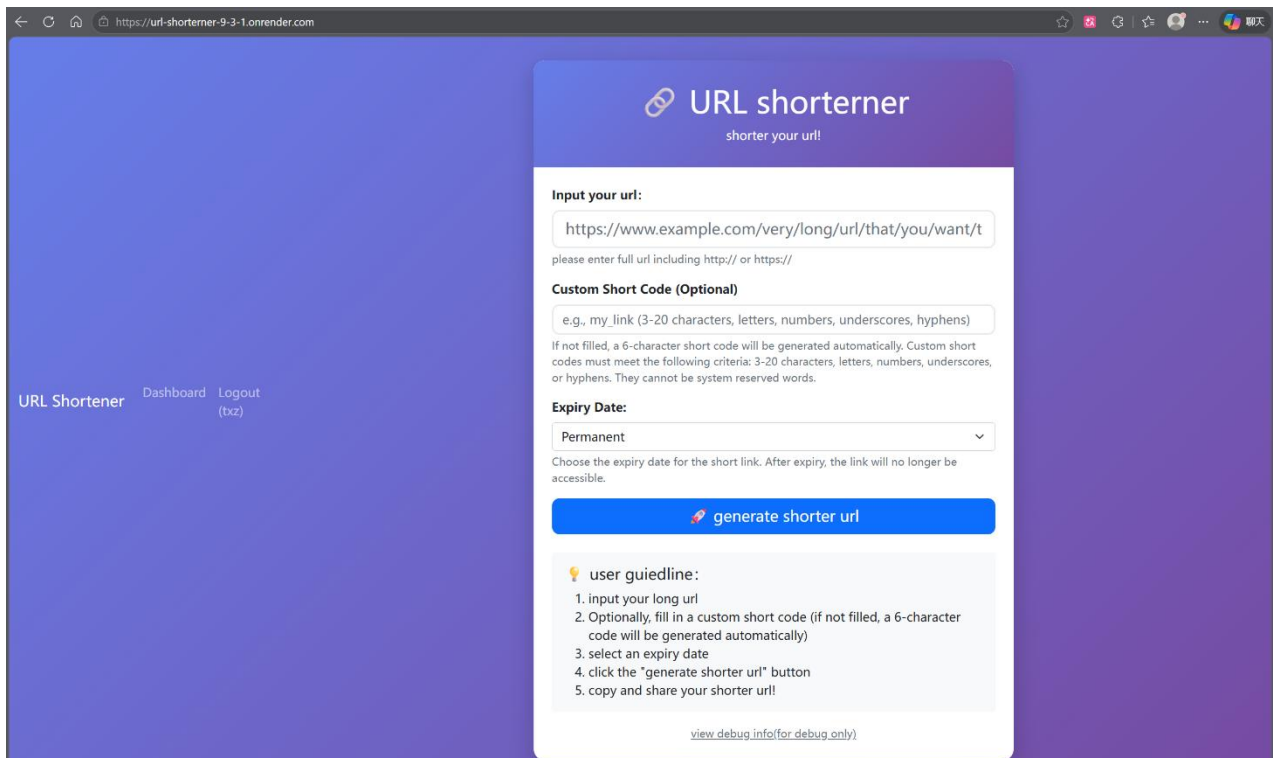


2.2 URL Shortening Core Engine

- Long URL Submission: Users submit a target URL. The system automatically prepends `https://` if no scheme is provided.
- Random Short Code Generation: When no custom code is requested, the system generates a 6-character alphanumeric code using a cryptographically secure random selection (`random.choice`). The generator checks for collisions and reserved words, retrying up to 10 times before failing gracefully.
- Custom Short Code Support: Users may specify a vanity short code. The system enforces strict validation rules:
 - a) Length: 3–20 characters.
 - b) Allowed characters: a-z, A-Z, 0-9, `_`, `-`.
 - c) No leading/trailing hyphens or consecutive hyphens.
 - d) Prohibited use of reserved system paths (e.g., `admin`, `api`, `login`).
- Duplicate Prevention: The system verifies the uniqueness of any custom code against the database before insertion.

2.3 Expiration & Lifecycle Management

- **Configurable Expiry:** Users may select an expiration period for each shortened link: 1 day, 7 days, 30 days, or "Permanent".
- **Expiry Enforcement:** During redirection, the system compares the current timestamp against the stored `expires_at` value. Expired links return an HTTP 410 (Gone) error page.



The screenshot shows a web browser window with the URL `https://url-shortener-9-3-1.onrender.com`. The page has a purple gradient background. On the left, there is a sidebar with the text "URL Shortener" and links for "Dashboard" and "Logout (txz)". The main content area features a white card with the "URL shortener" logo and the tagline "shorter your url!". Below this, there is a form with the following fields and options:

- Input your url:** A text input field containing `https://www.example.com/very/long/url/that/you/want/t`. Below the input, a note says "please enter full url including http:// or https://".
- Custom Short Code (Optional):** A text input field containing `e.g., my_link`. A note below says "e.g., my_link (3-20 characters, letters, numbers, underscores, hyphens)". A further note states: "If not filled, a 6-character short code will be generated automatically. Custom short codes must meet the following criteria: 3-20 characters, letters, numbers, underscores, or hyphens. They cannot be system reserved words."
- Expiry Date:** A dropdown menu currently set to "Permanent". A note below says "Choose the expiry date for the short link. After expiry, the link will no longer be accessible."

Below the form is a blue button labeled "generate shorter url". At the bottom of the card, there is a "user guideline" section with a list of 5 steps: 1. input your long url, 2. Optionally, fill in a custom short code (if not filled, a 6-character code will be generated automatically), 3. select an expiry date, 4. click the "generate shorter url" button, 5. copy and share your shorter url!. A link "view debug info(for debug only)" is also present.

2.4 Redirection & Click Analytics

- **HTTP 302 Redirect:** Accessing a valid short code (`/<short_code>`) seamlessly redirects the user's browser to the original long URL.
- **Real-time Click Counter:** Each redirect increments the `click_count` field in the `url_mappings` table.
- **Detailed Click Logging:** The following metadata is recorded in the `clicks` table for every access:
 - a) **IP Address:** Extracted from X-Forwarded-For header (to support proxy deployments like Render) or `request.remote_addr`.
 - b) **User-Agent:** The full browser/client identifier.
 - c) **Referer:** The HTTP referrer, if available.

d) Timestamp: Exact date and time of access.

2.5 User Dashboard & Link Management

- **Personal Link Inventory:** Authenticated users can view all URLs they have created via /dashboard. The table displays the short code, original URL (truncated), click count, creation date, expiration date, and current status (Active/Expired).
- **Client-side Time Localization:** UTC timestamps from the database are converted to the user's local time zone via JavaScript, improving readability.
- **Link Deletion:** Users may delete any link they own. The action cascades to remove associated click logs and the corresponding QR code image file.
- **Lifecycle Extension:** For links with a set expiration date, users can extend the validity by an additional 30 days with a single click (/extend/<short code>).



txz's Short Links

[Create New Link](#)
[Logout](#)

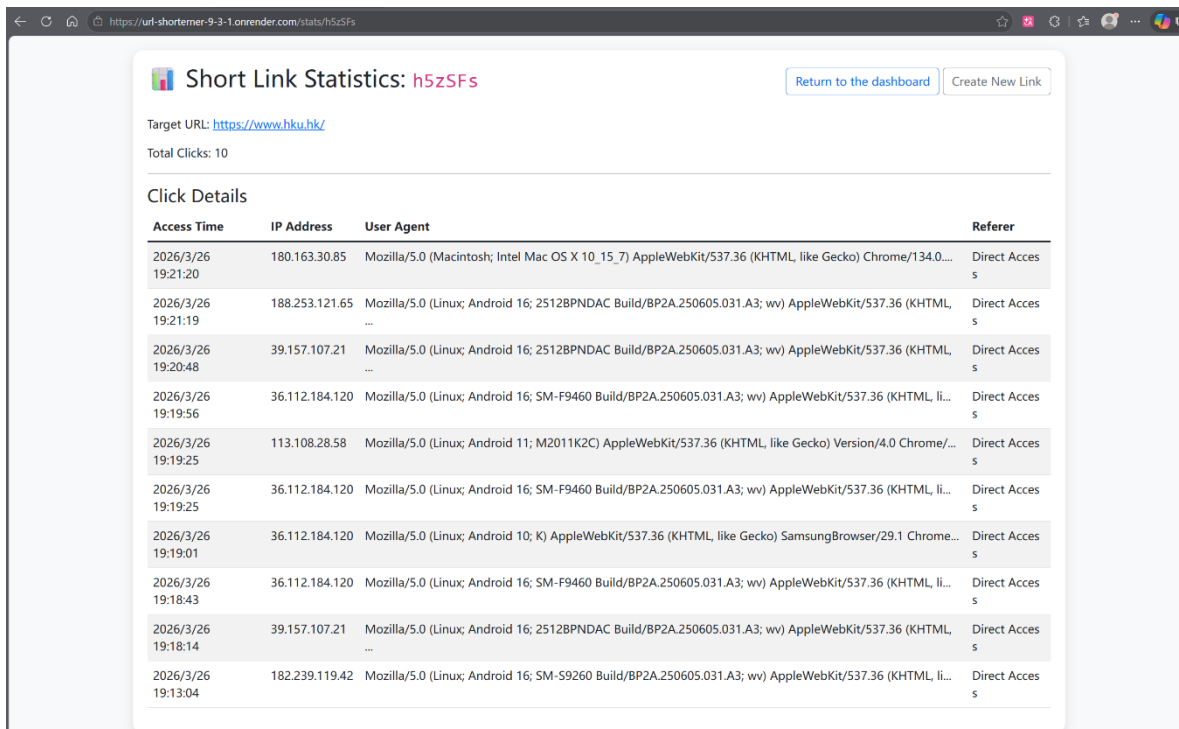
My Short Links

Short Code	Short URL	QR Code	Original URL	Clicks	Created At	Expires At	Status	Actions
h5z5Fs	https://url-shortener-r-9-3-1.onrender.com/h5z5Fs 	No QR	https://www.hku.hk/	10	2026/3/26 19:11:58	Permanent	Active	Delete Stats
r1yD54	https://url-shortener-r-9-3-1.onrender.com/r1yD54 	No QR	https://www.msn.com/zh-hk/health/other/%u	1	2026/3/26 19:08:48	Permanent	Active	Delete Stats
HBbQnw	https://url-shortener-r-9-3-1.onrender.com/m/HBbQnw 	No QR	https://www.guanben.io/n/tijayifei-bing_	1	2026/3/26 19:06:59	Permanent	Active	Delete Stats
y2UL3O	https://url-shortener-r-9-3-1.onrender.com/m/y2UL3O 	No QR	https://www.msn.com/zh-hk/news/world/%E4_	3	2026/3/26 17:37:39	Permanent	Active	Delete Stats
OYqq9p	https://url-shortener-r-9-3-1.onrender.com/m/OYqq9p 	No QR	https://www.guanben.io/n/tijayifei-bing_	0	2026/3/26 17:05:12	Permanent	Active	Delete Stats
yZXj8d	https://url-shortener-r-9-3-1.onrender.com/m/yZXj8d 	No QR	https://www.guanben.io/n/tijayifei-bing_	3	2026/3/26 16:05:46	Permanent	Active	Delete Stats
3XL9VQ	https://url-shortener-r-9-3-1.onrender.com/m/3XL9VQ 	No QR	https://www.guanben.io/n/tijayifei-bing_	0	2026/3/26 16:05:22	Permanent	Active	Delete Stats
DkgnNx	https://url-shortener-r-9-3-1.onrender.com/m/DkgnNx 	No QR	https://www.guanben.io/n/tijayifei-bing_	3	2026/3/26 15:55:22	Permanent	Active	Delete Stats

2.6 Detailed Statistics View

- Per-Link Analytics: Owners (and administrators) can access a dedicated statistics page (/stats/<short code>) that lists every individual click event in reverse

chronological order, including IP, User-Agent, timestamp, and referrer.



Short Link Statistics: h5zSFs

Target URL: <https://www.hku.hk/>

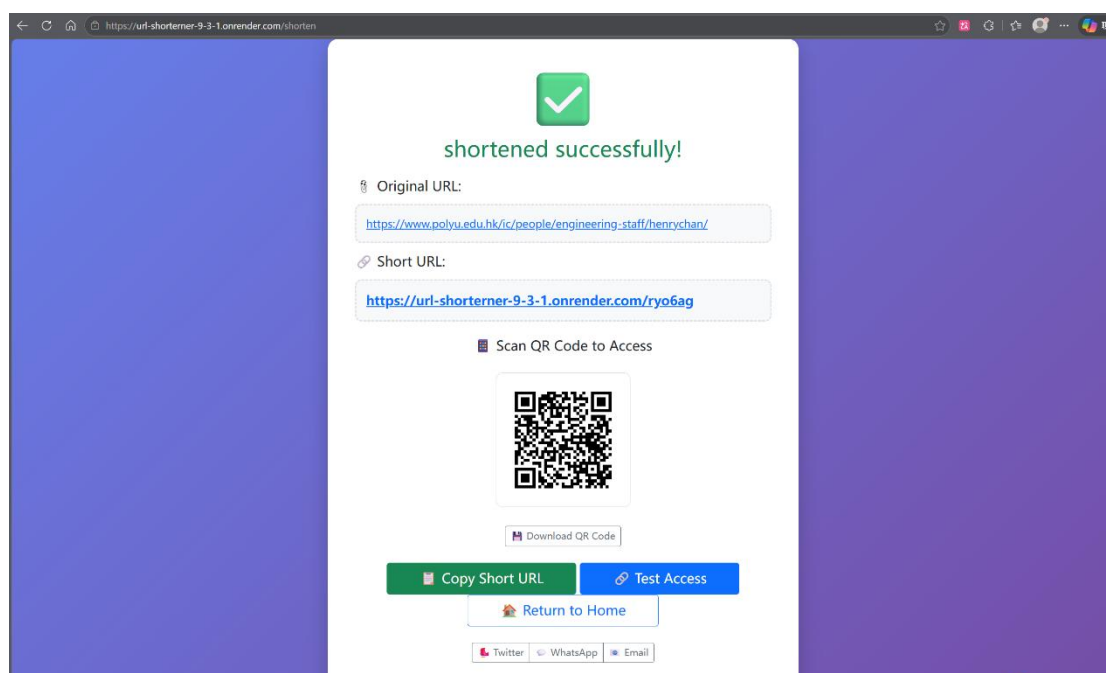
Total Clicks: 10

Click Details

Access Time	IP Address	User Agent	Referer
2026/3/26 19:21:20	180.163.30.85	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/134.0...	Direct Access
2026/3/26 19:21:19	188.253.121.65	Mozilla/5.0 (Linux; Android 16; 2512BPNDAC Build/BP2A.250605.031.A3; ww) AppleWebKit/537.36 (KHTML, ...	Direct Access
2026/3/26 19:20:48	39.157.107.21	Mozilla/5.0 (Linux; Android 16; 2512BPNDAC Build/BP2A.250605.031.A3; ww) AppleWebKit/537.36 (KHTML, ...	Direct Access
2026/3/26 19:19:56	36.112.184.120	Mozilla/5.0 (Linux; Android 16; SM-F9460 Build/BP2A.250605.031.A3; ww) AppleWebKit/537.36 (KHTML, li...	Direct Access
2026/3/26 19:19:25	113.108.28.58	Mozilla/5.0 (Linux; Android 11; M2011K2C) AppleWebKit/537.36 (KHTML, like Gecko) Version/4.0 Chrome/...	Direct Access
2026/3/26 19:19:25	36.112.184.120	Mozilla/5.0 (Linux; Android 16; SM-F9460 Build/BP2A.250605.031.A3; ww) AppleWebKit/537.36 (KHTML, li...	Direct Access
2026/3/26 19:19:01	36.112.184.120	Mozilla/5.0 (Linux; Android 10; K) AppleWebKit/537.36 (KHTML, like Gecko) SamsungBrowser/29.1 Chrome...	Direct Access
2026/3/26 19:18:43	36.112.184.120	Mozilla/5.0 (Linux; Android 16; SM-F9460 Build/BP2A.250605.031.A3; ww) AppleWebKit/537.36 (KHTML, li...	Direct Access
2026/3/26 19:18:14	39.157.107.21	Mozilla/5.0 (Linux; Android 16; 2512BPNDAC Build/BP2A.250605.031.A3; ww) AppleWebKit/537.36 (KHTML, ...	Direct Access
2026/3/26 19:13:04	182.239.119.42	Mozilla/5.0 (Linux; Android 16; SM-S9260 Build/BP2A.250605.031.A3; ww) AppleWebKit/537.36 (KHTML, li...	Direct Access

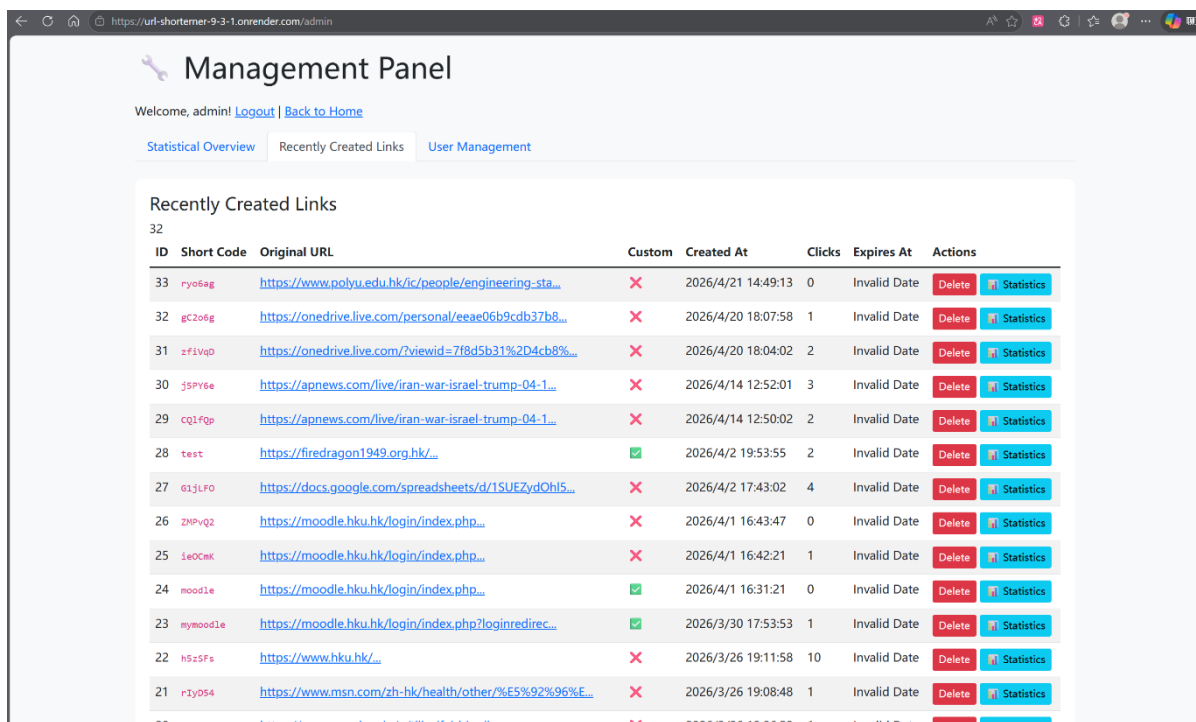
2.7 QR Code Generation

- **Automatic Generation:** Upon successful creation of a short URL, a QR code encoding the short link is generated using the qrcode library and saved as a PNG file in static/qrcodes/.
- **Frontend Integration:** The success page displays the QR code image (if generation succeeded) and provides a download button for users to save the image locally.



2.8 Administrative Panel

- Admin Authentication: Access to /admin is restricted to the user with username admin (hardcoded for the prototype phase).
- Global Statistics Overview: The admin dashboard displays:
 - Total number of shortened URLs.
 - Number of custom short codes.
 - Aggregate click count across all links.
- Recent Links Monitoring: A table listing the 20 most recently created links, with options to delete any link or view its statistics.
- User Management: A complete list of registered users with the ability for the admin to delete any user account (excluding themselves). Deleting a user cascades to remove all their associated URLs.



The screenshot shows the 'Management Panel' with a navigation bar containing 'Statistical Overview', 'Recently Created Links', and 'User Management'. The 'Recently Created Links' section displays a table with 20 rows of link data. Each row includes an ID, Short Code, Original URL, Custom status, Created At, Clicks, Expires At, and Actions (Delete and Statistics).

ID	Short Code	Original URL	Custom	Created At	Clicks	Expires At	Actions
33	ry06ag	https://www.polyu.edu.hk/ic/people/engineering-sta...	✗	2026/4/21 14:49:13	0	Invalid Date	Delete Statistics
32	gC2o6g	https://onedrive.live.com/personal/eeae06b9cdb37b8...	✗	2026/4/20 18:07:58	1	Invalid Date	Delete Statistics
31	zF1VqD	https://onedrive.live.com/?viewid=7f8d5b31%2D4cb8%	✗	2026/4/20 18:04:02	2	Invalid Date	Delete Statistics
30	jSPY6e	https://apnews.com/live/iran-war-israel-trump-04-1...	✗	2026/4/14 12:52:01	3	Invalid Date	Delete Statistics
29	cQ1FqP	https://apnews.com/live/iran-war-israel-trump-04-1...	✗	2026/4/14 12:50:02	2	Invalid Date	Delete Statistics
28	test	https://firedragon1949.org.hk/...	✓	2026/4/2 19:53:55	2	Invalid Date	Delete Statistics
27	61jLF0	https://docs.google.com/spreadsheets/d/1SUfZydOhj5...	✗	2026/4/2 17:43:02	4	Invalid Date	Delete Statistics
26	ZHPvQ2	https://moodle.hku.hk/login/index.php...	✗	2026/4/1 16:43:47	0	Invalid Date	Delete Statistics
25	ieOCmK	https://moodle.hku.hk/login/index.php...	✗	2026/4/1 16:42:21	1	Invalid Date	Delete Statistics
24	moodle	https://moodle.hku.hk/login/index.php...	✓	2026/4/1 16:31:21	0	Invalid Date	Delete Statistics
23	mymoodle	https://moodle.hku.hk/login/index.php?loginredirec...	✓	2026/3/30 17:53:53	1	Invalid Date	Delete Statistics
22	h5z5Fs	https://www.hku.hk/...	✗	2026/3/26 19:11:58	10	Invalid Date	Delete Statistics
21	r3y054	https://www.msn.com/zh-hk/health/other/%E5%92%96%E...	✗	2026/3/26 19:08:48	1	Invalid Date	Delete Statistics
20	...	https://www.msn.com/zh-hk/health/other/%E5%92%96%E...	✗	2026/3/26 19:06:50	1	Invalid Date	Delete Statistics

2.9 Cross-Database Compatibility

- Dual Database Support: The application seamlessly operates with either SQLite (for local development) or PostgreSQL (for production deployment), toggled by the presence of the DATABASE_URL environment variable.
- Automated Schema Migration: The init_db() function detects the active database engine and automatically creates tables. Additionally, it checks for the existence

of required columns and adds any missing columns

(e.g., `is_custom`, `click_count`, `expires_at`) to ensure compatibility with older schema versions.

2.10 Frontend User Interface

- **Responsive Design:** All HTML templates (`index.html`, `login.html`, `register.html`, `dashboard.html`, `link_stats.html`, `admin_dashboard.html`, `forgot_password.html`, `reset_password.html`) are styled with Bootstrap 5, ensuring a consistent and mobile-friendly user experience.
- **Social Sharing & Clipboard Integration:** The success page includes buttons to copy the short URL to the clipboard and to share the link directly via Twitter, WhatsApp, or Email using pre-filled message templates.
- **Error & Success Handling:** Custom, visually distinct HTML pages are returned for errors (e.g., invalid custom code, link not found, expired link) and successful URL creation, providing clear feedback and navigation options.

2.11 Deployment Readiness

- **Gunicorn Configuration:** A `gunicorn_config.py` file is prepared for production WSGI serving.
- **Dependency Management:** `requirements.txt` lists all necessary Python packages.
- **Environment Variable Configuration:** The application is configured to read `PORT`, `SECRET_KEY`, and `DATABASE_URL` from the environment, aligning with best practices for cloud platforms like Render.

3. Methodology & Implementation

3.1 Database Design

The application employs a relational database model with three primary tables: `users`, `url_mappings`, and `clicks`. The system supports both SQLite (for local development) and PostgreSQL (for production deployment) through a unified connection factory that toggles SQL syntax and placeholder styles (`?` vs. `%s`) based on the `DATABASE_URL` environment variable.

i. Users Table

The users table stores account credentials and password recovery tokens. Passwords are never stored in plaintext; instead, a salted hash generated by Werkzeug's `generate_password_hash` (using the `scrypt` algorithm by default) is persisted.

Column	Type	Description
id	SERIAL / INTEGER	Auto-incrementing unique identifier.
username	VARCHAR(100) / TEXT	User's chosen display name.
email	VARCHAR(255) / TEXT	Email address for password recovery.
password_hash	VARCHAR(255) / TEXT	Hashed password using Werkzeug security utilities.
create_at	TIMESTAMP	Account registration timestamp.
reset_token	VARCHAR(36) / TEXT	UUID v4 token for password reset requests.
reset_token_expiry	TIMESTAMP	Expiration timestamp for the reset token (valid for 1 hour).

ii. URL Mappings Table

The `url_mappings` table is the core of the shortening service. Each row represents a single shortened link and contains both the mapping information and metadata for lifecycle management.

Column	Type	Description
id	SERIAL / INTEGER	Auto-incrementing record ID.
long_url	TEXT	The original, full-length destination URL.
short_code	VARCHAR(50) / TEXT	The unique identifier used in the shortened path (e.g., <code>abc123</code>).

is_custom	BOOLEAN	Indicates whether the short_code was user-defined.
created_at	TIMESTAMP	Creation timestamp of the mapping.
click_count	INTEGER	Cumulative number of times the short link has been accessed.
user_id	INTEGER	The owner of the link. Set to NULL if the user is deleted, preserving link data.
expires_at	TIMESTAMP	The date and time after which the link becomes invalid. NULL indicates no expiration.

A unique index on short_code is enforced by the UNIQUE constraint, ensuring $O(\log n)$ lookup performance during redirection. The user_id foreign key establishes referential integrity and allows cascading operations (e.g., retrieving all links belonging to a specific user).

iii. Clicks Table

The clicks table logs detailed metadata for every access event. This normalized schema separates transactional analytics from the primary mapping table, preventing write contention on url_mappings and enabling granular analysis without impacting redirection latency.

Column	Type	Description
id	SERIAL / INTEGER	Auto-incrementing log entry ID.
short_code	VARCHAR(50) / TEXT	The short code that was accessed.
ip_address	VARCHAR(45) / TEXT	Client IP address (supports IPv6 length).
user_agent	TEXT	The User-Agent header string from the client.
accessed_at	TIMESTAMP	Precise time of the click event.
referrer	TEXT	The HTTP Referrer header, indicating the source of the click.

A non-unique index is created on `short_code` to accelerate queries that retrieve click history for a specific link (e.g., the `/stats` page). The `ON DELETE CASCADE` clause ensures that click logs are automatically purged when the associated short link is removed, maintaining data consistency.

iv. Schema Migration & Compatibility

To support iterative development and deployment across different database engines, the `init_db()` function implements a self-healing migration mechanism. It first executes the standard `CREATE TABLE IF NOT EXISTS` statements for all three tables. Subsequently, it queries the database's information schema (PostgreSQL) or `PRAGMA table_info` (SQLite) to detect missing columns. If a column such as `is_custom` or `expires_at` is absent from an existing database, an `ALTER TABLE` statement is dynamically issued to add the column. This approach ensures that the application can be deployed over an older database version without manual migration scripts.

3.2 Short Code Generation & Validation Algorithm

The system provides two pathways for obtaining a short code: automatic random generation and user-specified custom codes. Both pathways are governed by a rigorous validation framework to ensure URL safety and system integrity.

i. Automatic Random Code Generation

When a user does not provide a custom code, the system generates a 6-character alphanumeric string. The generation algorithm is defined as follows:

```
import random
import string

def generate_short_code(length=6):
    characters = string.ascii_letters + string.digits
    return "".join(random.choice(characters) for _ in range(length))
```

The character set consists of 26 lowercase letters, 26 uppercase letters, and 10 digits, yielding a total of 62 possible characters. A 6-character code therefore provides a theoretical keyspace of $62^6 \approx 5.68 \times 10^{10}$ (approximately 56.8 billion) unique

combinations[3].

Conflict Handling: Before inserting the generated code, the system performs a database lookup to verify its uniqueness. If a conflict occurs (code already exists), the generator will retry up to 10 times. Due to the large key space, the probability of a conflict is low when the number of existing entries is small; however, the retry loop provides a reliable fallback mechanism. If all 10 attempts result in a conflict, the system will return an error, prompting the user to retry or use custom code.

Reserved Word Filtering: In addition to database uniqueness, the generated code is compared against a predefined list of `RESERVED_WORDS`. This list contains route endpoints (e.g., login, admin, api, dashboard) and function paths that may conflict with the application's URL structure. Any generated code that matches a reserved word (case-insensitive) is discarded, and new code is generated. This pre-filtering mechanism prevents routing ambiguity and potential security bypasses.

ii. Custom Code Validation

User-submitted custom codes are subject to a comprehensive validation function, `validate_custom_code(code)`, which enforces the following rules:

Rule	Implementation	Rationale
Length between 3 and 20 characters	if <code>len(code) < 3</code> or <code>len(code) > 20</code>	Prevents excessively short (easy to brute-force) or long (defeating the purpose of shortening) codes.
Allowed character set: a-zA-Z0-9_-	Regular expression <code>^[a-zA-Z0-9_-]+\$</code>	Ensures URL safety and avoids percent-encoding issues in most contexts.
No leading or trailing hyphens	if <code>code.startswith('-')</code> or <code>code.endswith('-')</code>	Prevents ambiguity in domain name concatenation and

		improves readability.
No consecutive hyphens	if '--' in code	Prevents visual confusion and potential security issues related to IDN homograph attacks.
Not a reserved word	if code.lower() in RESERVED_WORDS	Protects application routes from being hijacked or overridden.

If all validations pass and the custom code does not already exist in the `'url_mappings'` table, the code is accepted and stored, and `'is_custom'` is set to `'True'`. Otherwise, a descriptive error message is returned to the user indicating which rule was violated.

iii. Expiration Logic

The system supports four expiration options: 1 day, 7 days, 30 days, and indefinite. The corresponding time difference objects are stored in the `'EXPIRY_OPTIONS'` dictionary. When creating a link, for options with a time limit, the expiration time field is set to the current time plus the corresponding time difference; for indefinite links, this field is set to `NULL`.

During redirection, the system retrieves the expiration time value. If this value is not `NULL` and the current time has exceeded the expiration time, the system returns an HTTP 410 (Resource expired) error page. This verification operation is performed before incrementing click counts or logging access, ensuring that expired links do not consume analytics resources.

iv. Redirection and Click Logging Workflow

The redirection process is optimized for minimal latency while capturing comprehensive analytics. The sequence of operations is as follows:

- a) **Lookup:** The `short_code` is used to query `url_mappings`. The unique index ensures a fast point lookup.
- b) **Expiry Check:** If the link is expired, abort with a 410 error.

- c) Click Count Increment: An `UPDATE url_mappings SET click_count = click_count + 1` is executed.
- d) Metadata Extraction: The client IP is extracted from the X-Forwarded-For header (if present) to accommodate reverse proxy setups (e.g., Render, Nginx). The User-Agent and Referer headers are also captured.
- e) Log Insertion: A new row is inserted into the clicks table with the gathered metadata.
- f) Redirection: An HTTP 302 Found response is returned with the Location header set to the long_url.

All database operations are performed within a single request context. While this synchronous approach is sufficient for the current prototype scale, the architecture allows for future offloading of the click logging step to an asynchronous task queue (e.g., Redis Queue) to further reduce tail latency under high concurrency.

3.3 QR Code Integration

QR code generation is implemented using the qrcode library. Upon successful creation of a short link, the system asynchronously (in the same request thread) generates a PNG image encoding the full short URL. The image is saved to `static/qrcodes/{short_code}.png`. The success page then conditionally renders an `<img>` tag if the file exists. This approach decouples the generation process from the HTTP response; if file I/O fails, the user still receives a functional success page without the QR code, ensuring graceful degradation[4].

4. Critical Evaluation

This section provides an objective assessment of the project's current status from two main dimensions: security and usability. By identifying existing limitations and shortcomings, it also points to a clear direction for subsequent optimization work in the final stages of development.

4.1 Security Weaknesses

Although the project implements salted cryptographic hashing using industry-standard

algorithms, several significant security vulnerabilities remain unresolved in the current prototype. These issues must be addressed before the system can be considered suitable for production deployment.

- **Vulnerable Administrator Privilege Determination:** Currently, administrator privileges are granted entirely based on hard-coded string comparisons, specifically checking if the session username equals "admin". This method has two fundamental flaws: First, any ordinary user who successfully registers with the username "admin" automatically gains full administrative privileges over the entire system. Second, the database schema lacks a dedicated "is_admin" flag, meaning that privilege escalation or delegation cannot be performed without modifying the source code.
- **Cross-Site Request Forgery (CSRF) Vulnerability:** All state-changing POST requests (including link deletion, expiration extension, password reset submission, and administrator user removal) are not protected against CSRF tokens. Therefore, a malicious third-party website could construct a hidden form that, when accessed by an authenticated user, silently triggers an unexpected action on a URL shortening service. Given that this application manages user resources and sensitive account operations, the lack of CSRF protection poses a real security risk, potentially leading to unauthorized data modification or account disclosure.
- **The password reset workflow is incomplete:** the current password recovery mechanism generates a valid reset token and displays the corresponding reset link directly in the server console output. While this approach is convenient for demonstrations and local testing, it completely bypasses the crucial step of sending the token to a verified email address of a legitimate user. In a real-world deployment, this renders the password reset function unusable, forcing administrators to rely on manual intervention to recover accounts. To improve this functionality, integration with an SMTP service or transactional email API is required.
- **Privacy issues in click logs:** The `clicks` table retains the full, unedited IP

address and User-Agent string for each user who accesses the shortened links. While this data is valuable for analytics and abuse detection, retaining it in plaintext indefinitely poses potential privacy risks. If the database is compromised, user access patterns and device fingerprints could be exposed. Implementing data anonymization techniques (such as IP address hashing or truncation) or developing data retention policies and automatically purging expired logs can mitigate this issue.

- **Lack of rate limiting:** The current system does not limit the frequency of requests to sensitive endpoints, including login attempts, registration submissions, and shortcode generation. This lack of rate limiting makes the application vulnerable to brute-force attacks targeting user credentials, as well as denial-of-service attacks launched by malicious attackers who exhaust system resources by submitting a large number of URL shortening requests. Introducing rate limiting based on IP or account will significantly enhance the system's ability to resist automated abuse.

4.2 Usability and Functional Gaps

From a user experience and usability perspective, the project already possesses a solid foundation of core functionalities. However, some improvements will enhance the value of the service and refine the overall experience.

- **Lack of visual analytics:** The link statistics page currently only presents click data in tabular form, listing each access event and its associated timestamp, IP address, and user agent string. While this raw data is comprehensive, it doesn't provide users with a clear overview of the data's composition. Adding visual charts, such as line graphs depicting click volume over time, IP geolocation charts, or pie charts showing device and browser distribution, would allow users to more effectively extract the information they need from the data.
- **Short link management is limited:** Once a short link is created, users cannot modify the target URL or associated shortcode. If a user accidentally enters an incorrect long URL, or later wants to update the target address without

breaking existing shared links, the only way is to delete the original link and create a new one. This operation results in the permanent loss of accumulated click statistics and interrupts all previously distributed short links. Providing an interface to edit the target URL would greatly improve operational flexibility.

- Batch operations are not supported: The user control panel currently only allows single operations on links, such as deleting or extending links one by one. This granular approach is extremely inefficient for users managing dozens or even hundreds of short links. The lack of batch deletion, batch expiration updates, or data export functionality (such as exporting to CSV format) limits the scalability of the management interface and causes considerable inconvenience to users.
- Lack of a public API: All system functions are accessible only through a browser-based web interface. The system does not provide programmatic interaction via API, thus preventing integration with third-party applications, command-line tools, or automated workflows. For developers and organizations needing to integrate URL shortening functionality into their software, the lack of an available API is a major obstacle to using this feature.
- Lack of expiration notification mechanism: Users who set limited expiration times for short links do not receive any advance notice when the link is about to expire. Therefore, links embedded in social media posts, email marketing campaigns, or documents may become invalid without the owner's knowledge, potentially disrupting user traffic and reducing user trust in the service. Setting up an automated email reminder system (e.g., triggering 48 hours before expiration) would proactively notify users and allow them to extend the link's validity before service disruptions.

4.3 Summary

In summary, the project has successfully achieved its functional goals within its established scope, resulting in a usable and feature-rich URL shortening platform.

However, the current implementation contains significant security vulnerabilities—particularly in administrator privilege management and CSRF protection—making it unsuitable for immediate deployment in a production environment. Meanwhile, several usability improvements, particularly in data visualization and link lifecycle management, will significantly enhance the user experience and strengthen the service's competitiveness.

5. Conclusion & Next Steps

5.1 Conclusion

During the mid-term evaluation phase, a fully functional prototype of the Flask-based URL shortening service was successfully delivered, encompassing features such as user authentication, custom shortcode generation, expiration management, click analytics, QR code generation, and management oversight. The system demonstrated cross-database compatibility between SQLite and PostgreSQL, enabling seamless switching between local development and cloud deployment environments. The core workflow, from URL submission to redirection and statistics retrieval, operated stably and reliably, and the responsive Bootstrap interface provided a user-friendly experience.

Several areas for further improvement have been identified, particularly in security hardening and usability enhancement. These findings will inform the final phase of development, which will focus on transforming the current prototype into a robust, production-ready application.

5.2 Next Steps

The existing tabular click logs will be updated with an interactive analytics dashboard, featuring core visualizations including:

i. Short Link Analytics Dashboard with Data Visualization

The existing tabular click log will be augmented with an interactive analytics dashboard. Key visualizations will include:

- Time-series line chart: displaying click data trends over daily, weekly, and

monthly .

- Geographic distribution map: Generated based on IP address geolocation, used to show the global coverage of short links.
- Device and browser segmentation: presented in the form of pie charts or donuts, allowing users to understand the technological environment of their audience.

This enhancement will transform raw click data into actionable business intelligence, aligning the project with industry-standard analytics platforms.

ii. Customizable Expiration Policies

The current expiration model—based solely on a fixed calendar date—will be extended to support composite conditions. Users will be permitted to define an expiration rule that triggers upon whichever criterion is met first:

- Calendar date (e.g., expires on December 31, 2026).
- Click count threshold (e.g., expires after 1,000 total accesses).

This dual-condition expiration mechanism provides granular control for marketing campaigns, limited-time promotions, or controlled-access scenarios.

iii. Password-Protected Short Links

A security feature will be introduced allowing users to optionally set a password for individual short links. Visitors attempting to access a protected link will be prompted to enter the correct password before redirection occurs. Implementation considerations include:

- Storing password hashes rather than plaintext in the `'url_mappings'` table.
- Designing a minimal, non-intrusive password entry interface.
- Supporting optional password hints or expiration of access grants.

This functionality addresses use cases involving sensitive internal documents, pre-release content, or private sharing among trusted parties.

iv. One-Click Click Report Generation

To facilitate offline analysis and record-keeping, users will be able to generate a comprehensive click report for any short link with a single button click. The report will be delivered as a downloadable PDF or CSV file and will include:

- Summary statistics (total clicks, unique IPs, top referrers).
- Complete click log with timestamps converted to the user's local time zone.
- Embedded QR code and short link metadata.

This feature enhances the service's utility for professional reporting and accountability.

v. Comprehensive Security Hardening

Based on the critical evaluation findings, the following security improvements will be implemented with high priority:

- Database-backed administrator roles: Replace the hardcoded `admin` username check with an `is\_admin` Boolean column in the `users` table, and restrict registration of the `admin` username.
- CSRF protection: Integrate Flask-WTF or an equivalent middleware to generate and validate CSRF tokens on all state-changing POST requests.
- Password reset email delivery: Integrate a transactional email service (e.g., SendGrid, Resend, or AWS SES) to securely transmit reset tokens to users' verified email addresses.
- Rate limiting: Apply per-IP and per-account request throttling on authentication endpoints and the URL shortening endpoint to mitigate brute-force and denial-of-service attacks.
- Click log privacy: Implement IP address hashing (e.g., SHA-256 with a rotating salt) and establish a configurable data retention policy for automated purging of aged click records.

vi. Trilingual User Interface (English, Traditional Chinese, Cantonese)

As this project serves as the Master's capstone for a student at the University of Hong Kong, the user interface will be localized to support three language variants reflecting the linguistic environment of Hong Kong:

- English (default and fallback).
- Simplified Chinese, widely used in formal documentation and signage.
- Cantonese, presented in written colloquial form to resonate with the local community.

The implementation will employ either Flask-Babel or a custom internationalization (i18n) solution based on gettext. All static text strings in HTML templates, form validation messages, and error notifications will be exported to the translation file. The language selector will be prominently displayed in the navigation bar, and the user's language preferences will be saved via session or cookie.

5.3 Anticipated Final Deliverables

Upon completion of the above tasks, this project will yield the following tangible deliverables:

- A fully secure, production-ready URL-shortened web application.
- A source code repository containing a complete commit history and inline documentation.
- Deployment configuration for a cloud platform (Render, Heroku, or a similar platform).
- User manuals and technical documentation.
- A final project report and presentation materials that meet the requirements of the University of Hong Kong Master's dissertation

- [1] *The New York Times*. "Mini-Links to Web Sites Are Multiplying." May 4, 2009.
- [2] Neumann, A., Barnickel, J., & Meyer, U. *Security and Privacy Implications of URL Shortening Services*. Proceedings of the Workshop on Web 2.0 Security and Privacy, 2010.
- [3] Gochhait, S., Rathore, Y. S., Leonova, I., Pandey, M. S., Saraswat, B. K., Maurya, S. K., Singh, H. R., & Bansal, N. *URL shortener for web consumption: an extensive and impressive security algorithm*. Indonesian Journal of Electrical Engineering and Computer Science (IJECS), 2024, 35(1), 284-291. DOI: 10.11591/ijeecs.v35.i1.pp284-291.
- [4] qrcode 7.4. *Pure Python QR Code Generator*. Python Package Index. [https://pypi.org/project/qrcode/\[reference:48\]](https://pypi.org/project/qrcode/[reference:48]).